

Reading a DS18B20 with STM32G030 Using TIM3 10 ms Interrupts

Practical Coding Walkthrough

1 Introduction

This document shows all essential coding steps to read a DS18B20 temperature sensor using an STM32G030 microcontroller. Timer 3 (TIM3) is configured to generate an interrupt every 10 ms, and the DS18B20 is read every 2 s from inside a simple state machine driven by that interrupt. The examples use STM32Cube HAL-style C code.

2 Hardware connections

2.1 Basic wiring

- DS18B20 **DQ** pin connected to STM32G030 pin **PA1** (example).
- DS18B20 **GND** to MCU GND.
- DS18B20 **VDD** to 3.3 V (normal powered mode).
- One 4.7 k Ω pull-up resistor from DQ (PA1) to 3.3 V.

You can change the GPIO pin later; only the pin definitions in the code need to be updated.

2.2 GPIO configuration idea

The DS18B20 uses an open-drain 1-Wire bus. We emulate this by:

- Configuring the pin as push-pull output when driving low.
- Reconfiguring as input (high-impedance) to release the line (pulled high by the resistor).

3 TIM3 configuration for 10 ms interrupts

Assume:

- System core clock: 16 MHz (HSI, default for many STM32G0 setups).
- TIM3 clock: 16 MHz.
- We want an update event every 10 ms.

Timer period:

$$T = \frac{(PSC + 1) \cdot (ARR + 1)}{f_{TIM}}$$

Choose:

- $PSC = 1599$
- $ARR = 99$

Then:

$$T = \frac{(1599 + 1) \cdot (99 + 1)}{16\,000\,000} = \frac{1600 \cdot 100}{16\,000\,000} = 0.01\,s = 10\,ms$$

3.1 TIM3 initialization code

```
1 // Global timer handle
2 TIM_HandleTypeDef htim3;
3
4 static void MX_TIM3_Init(void)
5 {
6     __HAL_RCC_TIM3_CLK_ENABLE();
7
8     htim3.Instance = TIM3;
9     htim3.Init.Prescaler      = 1599;    // PSC
10    htim3.Init.CounterMode    = TIM_COUNTERMODE_UP;
11    htim3.Init.Period         = 99;      // ARR
12    htim3.Init.ClockDivision  = TIM_CLOCKDIVISION_DIV1;
13    htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
14
15    if (HAL_TIM_Base_Init(&htim3) != HAL_OK)
16    {
17        Error_Handler();
18    }
19
20    // Enable update interrupt
21    HAL_NVIC_SetPriority(TIM3_IRQn, 1, 0);
22    HAL_NVIC_EnableIRQ(TIM3_IRQn);
23
24    if (HAL_TIM_Base_Start_IT(&htim3) != HAL_OK)
25    {
26        Error_Handler();
27    }
28 }
```

4 Minimal 1-Wire + DS18B20 driver

4.1 Pin abstraction

```
1 #include "stm32g0xx_hal.h"
2
3 // Adjust these to your actual pin choice
4 #define DS18B20_GPIO_PORT    GPIOA
5 #define DS18B20_GPIO_PIN     GPIO_PIN_1
6
7 // ROM commands
8 #define DS18B20_CMD_SKIP_ROM 0xCC
9 // Function commands
10 #define DS18B20_CMD_CONVERT_T 0x44
11 #define DS18B20_CMD_READ_SCR  0xBE
12
13 extern TIM_HandleTypeDef htim3; // if you want to use HAL_Delay or timer-based delays
14
15 static void DS18B20_DQ_Output(void)
16 {
17     GPIO_InitTypeDef GPIO_InitStruct = {0};
18     GPIO_InitStruct.Pin      = DS18B20_GPIO_PIN;
19     GPIO_InitStruct.Mode     = GPIO_MODE_OUTPUT_PP;
20     GPIO_InitStruct.Pull     = GPIO_NOPULL;
21     GPIO_InitStruct.Speed    = GPIO_SPEED_FREQ_HIGH;
22     HAL_GPIO_Init(DS18B20_GPIO_PORT, &GPIO_InitStruct);
23 }
24
25 static void DS18B20_DQ_Input(void)
26 {
27     GPIO_InitTypeDef GPIO_InitStruct = {0};
28     GPIO_InitStruct.Pin      = DS18B20_GPIO_PIN;
```

```

29  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
30  GPIO_InitStruct.Pull = GPIO_NOPULL;
31  HAL_GPIO_Init(DS18B20_GPIO_PORT, &GPIO_InitStruct);
32 }
33
34 static void DQ_WriteLow(void)
35 {
36     DS18B20_DQ_Output();
37     HAL_GPIO_WritePin(DS18B20_GPIO_PORT, DS18B20_GPIO_PIN, GPIO_PIN_RESET);
38 }
39
40 static void DQ_Release(void)
41 {
42     DS18B20_DQ_Input(); // line floats high via pull-up
43 }
44
45 static GPIO_PinState DQ_Read(void)
46 {
47     return HAL_GPIO_ReadPin(DS18B20_GPIO_PORT, DS18B20_GPIO_PIN);
48 }

```

4.2 Microsecond delay helper

For simplicity, we use a crude delay based on a busy loop. For production, use a timer or DWT.

```

1 static void delay_us(uint32_t us)
2 {
3     // crude approximation for 16 MHz core
4     // each loop ~4 cycles -> adjust as needed
5     uint32_t count = us * 4;
6     while (count--)
7     {
8         __NOP();
9     }
10 }

```

4.3 1-Wire reset and presence

```

1 static uint8_t OneWire_Reset(void)
2 {
3     uint8_t presence;
4
5     DQ_WriteLow();
6     delay_us(480); // reset low
7     DQ_Release();
8     delay_us(70); // wait before reading
9
10    presence = (DQ_Read() == GPIO_PIN_RESET) ? 1 : 0; // presence pulse
11
12    delay_us(410); // finish timeslot
13    return presence;
14 }

```

4.4 Write and read bit

```

1 static void OneWire_WriteBit(uint8_t bit)
2 {
3     DQ_WriteLow();
4     if (bit)
5     {
6         delay_us(6); // write '1' low time
7         DQ_Release();

```

```

8     delay_us(64);
9 }
10 else
11 {
12     delay_us(60);          // write '0' low time
13     DQ_Release();
14     delay_us(10);
15 }
16 }
17
18 static uint8_t OneWire_ReadBit(void)
19 {
20     uint8_t bit;
21
22     DQ_WriteLow();
23     delay_us(6);
24     DQ_Release();
25     delay_us(9);           // wait before sampling
26     bit = (DQ_Read() == GPIO_PIN_SET) ? 1 : 0;
27     delay_us(55);         // rest of timeslot
28
29     return bit;
30 }

```

4.5 Write and read byte

```

1 static void OneWire_WriteByte(uint8_t data)
2 {
3     for (uint8_t i = 0; i < 8; i++)
4     {
5         OneWire_WriteBit(data & 0x01);
6         data >>= 1;
7     }
8 }
9
10 static uint8_t OneWire_ReadByte(void)
11 {
12     uint8_t data = 0;
13
14     for (uint8_t i = 0; i < 8; i++)
15     {
16         uint8_t bit = OneWire_ReadBit();
17         data >>= 1;
18         if (bit)
19         {
20             data |= 0x80;
21         }
22     }
23     return data;
24 }

```

4.6 Start temperature conversion

```

1 static uint8_t DS18B20_StartConversion(void)
2 {
3     if (!OneWire_Reset())
4     {
5         return 0; // no presence
6     }
7
8     OneWire_WriteByte(DS18B20_CMD_SKIP_ROM);
9     OneWire_WriteByte(DS18B20_CMD_CONVERT_T);
10 }

```

```

11 // Conversion time depends on resolution; for 12-bit, up to 750 ms.
12 // We'll wait using our 2-second timer-based scheme, so no blocking here.
13 return 1;
14 }

```

4.7 Read scratchpad and convert to Celsius

```

1 static uint8_t DS18B20_ReadScratchpad(uint8_t *scratchpad, uint8_t len)
2 {
3     if (!OneWire_Reset())
4     {
5         return 0;
6     }
7
8     OneWire_WriteByte(DS18B20_CMD_SKIP_ROM);
9     OneWire_WriteByte(DS18B20_CMD_READ_SCR);
10
11     for (uint8_t i = 0; i < len; i++)
12     {
13         scratchpad[i] = OneWire_ReadByte();
14     }
15
16     return 1;
17 }
18
19 static float DS18B20_ConvertToCelsius(uint8_t *scratchpad)
20 {
21     int16_t raw = (scratchpad[1] << 8) | scratchpad[0];
22     // Default 12-bit resolution: LSB = 0.0625 C
23     float temp = (float)raw * 0.0625f;
24     return temp;
25 }

```

5 Integration with TIM3 interrupt (2 s period)

We use the 10 ms TIM3 interrupt to:

- Increment a counter.
- Every 200 ticks (2 s), start a DS18B20 conversion.
- After enough time for conversion (e.g. 750 ms), read the scratchpad.

For simplicity, we implement a small state machine.

5.1 Global variables and state machine

```

1 #include "main.h"
2
3 TIM_HandleTypeDef htim3;
4
5 // 10 ms tick counter
6 volatile uint32_t g_tick10ms = 0;
7
8 // DS18B20 state machine
9 typedef enum {
10     DS_STATE_IDLE = 0,
11     DS_STATE_START_CONV,
12     DS_STATE_WAIT_CONV,
13     DS_STATE_READ_TEMP
14 } DS_State_t;

```

```

15
16 volatile DS_State_t g_dsState = DS_STATE_IDLE;
17 volatile uint32_t   g_dsTimerTicks = 0;    // counts 10 ms ticks
18 volatile float      g_lastTemperatureC = 0.0f;
19 volatile uint8_t    g_dsError = 0;

```

5.2 TIM3 IRQ handler and callback

```

1 void TIM3_IRQHandler(void)
2 {
3     HAL_TIM_IRQHandler(&htim3);
4 }
5
6 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
7 {
8     if (htim->Instance == TIM3)
9     {
10         g_tick10ms++;
11
12         // State machine for DS18B20
13         switch (g_dsState)
14         {
15             case DS_STATE_IDLE:
16                 // Every 200 ticks (2 s), start a new conversion
17                 if ((g_tick10ms % 200) == 0)
18                 {
19                     if (DS18B20_StartConversion())
20                     {
21                         g_dsTimerTicks = 0;
22                         g_dsState = DS_STATE_WAIT_CONV;
23                     }
24                     else
25                     {
26                         g_dsError = 1;
27                     }
28                 }
29                 break;
30
31             case DS_STATE_WAIT_CONV:
32                 g_dsTimerTicks++;
33                 // Wait at least 750 ms for 12-bit conversion
34                 if (g_dsTimerTicks >= 75) // 75 * 10 ms = 750 ms
35                 {
36                     g_dsState = DS_STATE_READ_TEMP;
37                 }
38                 break;
39
40             case DS_STATE_READ_TEMP:
41                 {
42                     uint8_t scratchpad[9];
43                     if (DS18B20_ReadScratchpad(scratchpad, 9))
44                     {
45                         g_lastTemperatureC = DS18B20_ConvertToCelsius(scratchpad);
46                         g_dsError = 0;
47                     }
48                     else
49                     {
50                         g_dsError = 1;
51                     }
52                     g_dsState = DS_STATE_IDLE;
53                     break;
54                 }
55
56             default:

```

```

57     g_dsState = DS_STATE_IDLE;
58     break;
59 }
60 }
61 }

```

5.3 GPIO and clock initialization

```

1 static void MX_GPIO_Init(void)
2 {
3     __HAL_RCC_GPIOA_CLK_ENABLE();
4
5     GPIO_InitTypeDef GPIO_InitStruct = {0};
6
7     // Start with line released (input)
8     GPIO_InitStruct.Pin = DS18B20_GPIO_PIN;
9     GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
10    GPIO_InitStruct.Pull = GPIO_NOPULL;
11    HAL_GPIO_Init(DS18B20_GPIO_PORT, &GPIO_InitStruct);
12 }
13
14 void SystemClock_Config(void)
15 {
16     // Minimal example: configure HSI at 16 MHz
17     // Use CubeMX-generated code in a real project.
18     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
19     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
20
21     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
22     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
23     RCC_OscInitStruct.HSIDiv = RCC_HSI_DIV1;
24     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
25     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
26     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
27     {
28         Error_Handler();
29     }
30
31     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
32                                 | RCC_CLOCKTYPE_PCLK1;
33     RCC_ClkInitStruct.SYSClkSource = RCC_SYSCCLKSOURCE_HSI;
34     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCCLK_DIV1;
35     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
36
37     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
38     {
39         Error_Handler();
40     }
41 }

```

5.4 Example main.c

```

1 #include "stm32g0xx_hal.h"
2
3 // Forward declarations
4 void SystemClock_Config(void);
5 static void MX_GPIO_Init(void);
6 static void MX_TIM3_Init(void);
7
8 int main(void)
9 {
10     HAL_Init();
11     SystemClock_Config();

```

```

12  MX_GPIO_Init();
13  MX_TIM3_Init();
14
15  // Optionally, start first conversion immediately
16  g_dsState = DS_STATE_IDLE;
17  g_dsError = 0;
18
19  while (1)
20  {
21      // Main loop can use g_lastTemperatureC
22      // Example: simple polling or breakpoint inspection
23
24      if (g_dsError)
25      {
26          // handle error (e.g., blink LED, log, etc.)
27      }
28
29      // Small delay to reduce power / CPU usage
30      HAL_Delay(100);
31  }
32 }
33
34 void Error_Handler(void)
35 {
36     __disable_irq();
37     while (1)
38     {
39         // stay here
40     }
41 }

```

6 Adapting pins and timers

6.1 Changing the DS18B20 pin

To use another pin:

- Update DS18B20_GPIO_PORT and DS18B20_GPIO_PIN.
- Ensure the corresponding GPIO clock is enabled in MX_GPIO_Init.
- Adjust any board-specific constraints (e.g. if the pin is shared with other peripherals).

6.2 Changing the timer or interrupt period

To use another timer (e.g. TIM14) or a different period:

- Change the handle and instance in the timer init function.
- Recalculate PSC and ARR for the desired period.
- Update the IRQ handler name and NVIC configuration.
- Adjust the state machine tick counts (e.g. $2\text{ s} = 200\text{ ticks at }10\text{ ms}$).